TCP Server Experiment

TCP Server Experiment
- 1. TCP Server Overview
- 2. Core Concepts
  - 2.1 TCP Server Communication Model
  - 2.2 Configuration Parameters
  - 2.3 Core API
  - 2.4 Command Set
- 3. Hardware Dependencies
  - 3.1 Pin Mapping
- 4. Project Architecture and Flow
- 5. Source Code Explained
  - 5.1 Creating the Server Socket
  - 5.2 Accepting New Connections
  - 5.3 Polling Connected Clients
  - 5.4 Command Processing
- 6. Operating Procedures
  - 6.1 Environment Preparation
  - 6.2 Flashing and Running
  - 6.3 Expected Results
  - 6.4 Serial Output Example
- 7. Common Problems

1. TCP Server Overview

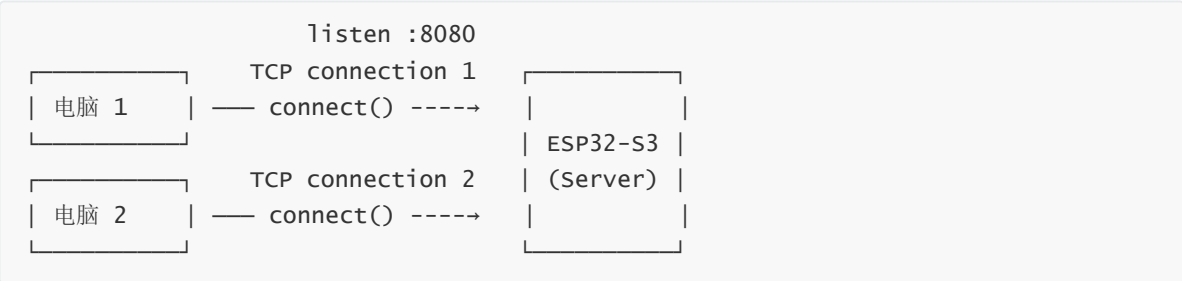
Unlike the client, **TCP (Transmission Control Protocol)** server listens on a specified port and waits for clients to actively connect. In this experiment the ESP32-S3 acts as a TCP Server, and the PC network debugging assistant connects as a Client, achieving one-to-many bidirectional send/receive — supporting up to 3 clients connecting simultaneously; the server can send a welcome message to each client and process commands.

Typical application scenarios:

Application Type	Example
Device management	Multiple PCs monitor one ESP32 device at the same time
HTTP server	Browser accesses the ESP32 web page
Data relay	Receives data from multiple clients and aggregates/forwards it

2. Core Concepts

2.1 TCP Server Communication Model



- `bind(("0.0.0.0", port))` binds the port; "0.0.0.0" means listening on all network interfaces
- `listen(n)` sets the maximum waiting queue
- `accept()` blocks waiting for a client connection and returns (conn, addr)

2.2 Configuration Parameters

Parameter	Description	Value Used in This Experiment
LISTEN_PORT	ESP32 listen port	`8080`
MAX_CLIENTS	Max simultaneous connections	`3`

2.3 Core API

```
import socket

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(("0.0.0.0", 8080))       # bind port
server.listen(3)                       # start listening
conn, addr = server.accept()           # accept client
conn.sendall(b"hello")                 # send to client
data = conn.recv(512)                  # receive from client
conn.close()                           # close client connection
```

2.4 Command Set

The client (PC network assistant) sends the following text commands; the ESP32 server dispatches them in `process_command()` after receiving them:

Command	Function	Reply Content
`PING`	Connectivity check	`PONG`
`TIME`	Query runtime status	Runtime (s) + free memory (KB)
`INFO`	Query device info	Free memory + total memory + usage + Flash capacity
Other	Echo	`Received: {original message}`

> Command matching is case-sensitive, fully consistent with the TCP client experiment.

3. Hardware Dependencies

Same as the TCP client — ESP32-S3 built-in WiFi + PC network assistant (TCP Client mode).

3.1 Pin Mapping

None — pure network communication using built-in WiFi.

4. Project Architecture and Flow

```
Script execution (single-file .py)
  |
  |- wifi_connect(): connect to WiFi (STA mode)
  |- create Server Socket -> bind(port) -> listen(3)
  |
  |- while True:
       |- accept(): non-blocking attempt to accept new connections
       |    |- new connection -> send welcome message -> add to clients[]
       |
       |- poll existing clients:
       |    |- recv_msg() -> process_command() -> send_msg()
       |    |- disconnected -> close() -> remove from clients[]
       |
       |- sleep(0.1s)
```

5. Source Code Explained

Full source code: `Source Code -> MicroPython -> 4.Network Protocols -> TCP_Server -> TCP_Server.py`

5.1 Creating the Server Socket

`bind("0.0.0.0", port)` listens on all network interfaces; `listen(3)` allows 3 queued connections; `settimeout(1)` makes `accept()` non-blocking.

```python
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.settimeout(1)                          # non-blocking accept

server.bind(("0.0.0.0", LISTEN_PORT))         # bind
server.listen(3)                                # start listening

ip = network.WLAN(network.STA_IF).ifconfig()[0]
print(f"[TCP] Listening on {ip}:{LISTEN_PORT}")
```

5.2 Accepting New Connections

`accept()` returns (conn, addr); new connections get a 0.5s timeout and are added to the clients list.

```
clients = []                                    # (conn, addr) list

while True:
    try:
        conn, addr = server.accept()            # non-blocking accept
        conn.settimeout(0.5)
        clients.append((conn, addr))
        print(f"[TCP] New client connected: {addr[0]}:{addr[1]}")
        send_msg(conn, f"Welcome to the ESP32-S3 server", addr)
    except OSError:
        pass                                    # timeout, no connection, continue
```

5.3 Polling Connected Clients

Iterates through all connected clients, checking each one with `recv` for incoming data. After disconnection, clients are removed from the list in reverse order to avoid index confusion.

```
dead_conns = []
for i, (conn, addr) in enumerate(clients):
    msg = recv_msg(conn, addr)
    if msg is False:                            # disconnected
        conn.close()
        dead_conns.append(i)
    elif msg:
        reply = process_command(msg)            # command dispatch
        if reply:
            send_msg(conn, reply, addr)

for i in reversed(dead_conns):                  # delete in reverse to avoid index
confusion
    del clients[i]
```

5.4 Command Processing

Same `process_command()` as the TCP client — `PING -> PONG`, `TIME -> runtime + memory`, `INFO -> device info`.

---

6. Operating Procedures

6.1 Environment Preparation

Thonny IDE steps: See `MicroPython -> 1. Before Development -> 3.Thonny IDE`.

1. Open the network debugging assistant 

on the PC, select **TCP Client** mode
2. The target IP is the ESP32-S3's IP (printed on the serial port), and the target port is `8080`

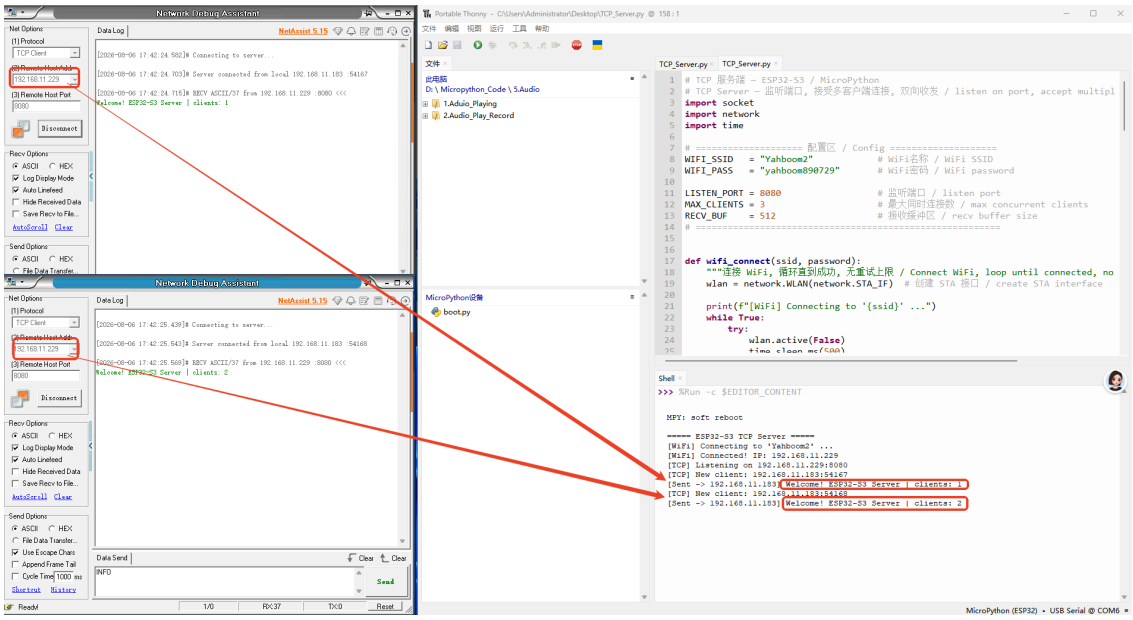
6.2 Flashing and Running

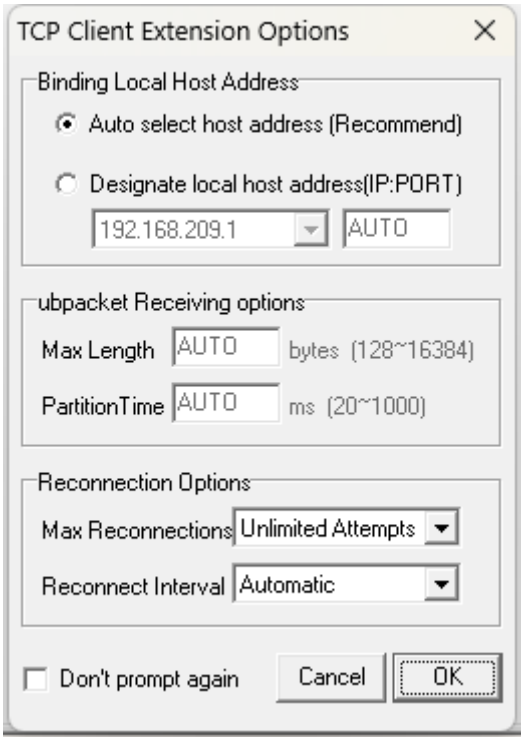
1. Modify `WIFI_SSID` / `WIFI_PASS` in the code

```
# ==================== 配置区 / Config ====================
WIFI_SSID    = "Yahboom2"              # WiFi名称 / WiFi SSID
WIFI_PASS    = "yahboom890729"         # WiFi密码 / WiFi password

LISTEN_PORT = 8080                     # 监听端口 / listen port
MAX_CLIENTS = 3                        # 最大同时连接数 / max concurrent clients
RECV_BUF    = 512                      # 接收缓冲区 / recv buffer size
# ----------------------------------------------------------
```

2. Run `TCP_Server.py` and note the IP printed on the serial port





6.3 Expected Results

After the PC network assistant connects to the ESP32, it receives a welcome message; sending `PING` returns `PONG`.

```
[2026-07-28 11:59:34.675]# SEND ASCII/4 >>>
PING

[2026-07-28 11:59:35.829]# RECV ASCII/4 from 192.168.11.229 :8080 <<<
PONG
```

6.4 Serial Output Example

```
===== ESP32-S3 TCP Server =====
[WiFi] Connecting to 'Yahboom2' ...
[WiFi] Connected! IP: 192.168.11.229
[TCP] Listening on 192.168.11.229:8080
[TCP] New client: 192.168.11.183:49837
[Sent -> 192.168.11.183] Welcome! ESP32-S3 Server | clients: 1
[TCP] New client: 192.168.11.183:49840
[Sent -> 192.168.11.183] Welcome! ESP32-S3 Server | clients: 2
[Recv <- 192.168.11.183] TIME
[Sent -> 192.168.11.183] ESP32-S3 status
  uptime: 360s
  Free memory: 8116 KB
[Recv <- 192.168.11.183] INFO
[Sent -> 192.168.11.183] ESP32-S3 Server
  Free memory: 8114 KB
  Total memory:   8127 KB
  Usage:   0.2%
  Flash:    16 MB
```


```
===== ESP32-S3 TCP Server =====
[WiFi] Connecting to 'Yahboom2' ...
[WiFi] Connected! IP: 192.168.11.229
[TCP] Listening on 192.168.11.229:8080
[TCP] New client: 192.168.11.183:49837
[Sent -> 192.168.11.183] Welcome! ESP32-S3 Server | clients: 1
[TCP] New client: 192.168.11.183:49840
[Sent -> 192.168.11.183] Welcome! ESP32-S3 Server | clients: 2
[Recv <- 192.168.11.183] TIME
[Sent -> 192.168.11.183] ESP32-S3 status
  uptime: 360s
  Free memory: 8116 KB
[Recv <- 192.168.11.183] INFO
[Sent -> 192.168.11.183] ESP32-S3 Server
  Free memory: 8114 KB
  Total memory:   8127 KB
  Usage:   0.2%
  Flash:    16 MB
```


7. Common Problems

Symptom	Cause	Solution
Client cannot connect	Firewall blocking or wrong IP	Check that the IP is the value printed by the ESP32, disable the firewall
`bind` fails	Port already in use	Use another port (e.g. 8081)
No log after client disconnects	`recv()` returns empty but disconnection is not detected	The code detects disconnection via `recv() == b""`, which is normal